Name: Abdulaziz Mohammed Alkhlawi
ID: 2240006095

N-Grams

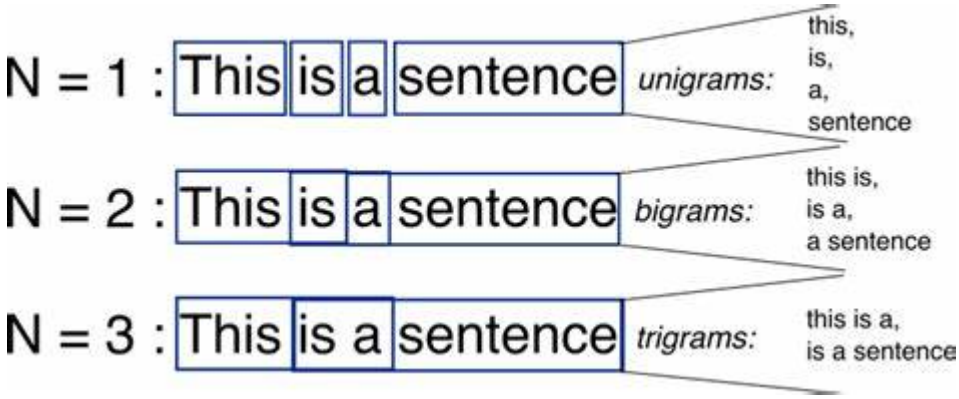
Completed lab solution. Original instructions: NLP_Lab3.

Run the cells from top to bottom. Results below were generated by executing this notebook. The `dataset` folder contains the original data needed by this lab. Missing Python packages are installed by the setup cell; pretrained model downloads need internet on their first run. Random seeds are fixed where supported.

```
# Install only missing dependencies into the active Jupyter Python environment.
import importlib.util, subprocess, sys, os, random
from pathlib import Path
packages = {'numpy': 'numpy', 'pandas': 'pandas', 'matplotlib': 'matplotlib', 'sklearn': 'scikit-learn',
'nltk': 'nltk', 'emoji': 'emoji'}
missing = [package for module, package in packages.items()
           if importlib.util.find_spec(module) is None]
if missing:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', *missing])
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['USE_TF'] = '0'  # Transformers examples use PyTorch.
os.environ['USE_FLAX'] = '0'
import numpy as np
import pandas as pd
from IPython.display import display, Markdown, HTML
random.seed(42)
np.random.seed(42)
pd.set_option('display.max_colwidth', 90)
pd.set_option('display.max_columns', 10)
DATA = Path('dataset')
assert DATA.exists() or 3 in (7, 8, 9), 'Open this notebook from its Lab folder.'
```

It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input. An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities.

Types of N-Grams



Use-cases of N-Grams

- Auto completion of sentences
- Auto spell check
- Voice-based personal assistant bots

Unigram

Unigram is an n-gram consisting of a single item of a sequence.

- It is commonly used to calculate the probability of finding a word in an article.

$$P(A) = \frac{\text{count of } A}{\text{count of all words}}$$

```
import nltk
for resource in ['punkt_tab', 'stopwords', 'wordnet', 'omw-1.4', 'averaged_perceptron_tagger_eng']:
    nltk.download(resource, quiet=True)
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

unigrams = ngrams(word_tokenize(string), 1)

for item in unigrams:
    print(item)
```

```
('moses',)
('supposes',)
('his',)
('toeses',)
('are',)
('roses',)
('but',)
('moses',)
('supposes',)
('erroneously',)
```

Calculating probabilities in unigram models

```
from collections import Counter
words_frq = sorted(Counter(word_tokenize(string)).items())
words_frq
```

```
[('are', 1),
 ('but', 1),
 ('erroneously', 1),
 ('his', 1),
 ('moses', 2),
 ('roses', 1),
 ('supposes', 2),
 ('toeses', 1)]
```

```
### Calculate probabilities:

probab = []

for count in range(len(words_frq)):
    pro = words_frq[count][1]/len(word_tokenize(string))
    probab.append((words_frq[count][0],pro))

print('The probabilities are: ', probab)
```

```
The probabilities are:  [('are', 0.1), ('but', 0.1), ('erroneously', 0.1), ('his', 0.1), ('moses', 0.2),
('roses', 0.1), ('supposes', 0.2), ('toeses', 0.1)]
```

Bigram

Bigram an n-gram consisting of two items of a sequence

- Two words that are used together to mean something specific

$$P(B \mid A) = \frac{\text{Count of } (AB)}{\text{Count of } (A)}$$

Method 1

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = nltk.bigrams(word_tokenize(string))
for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Method 2

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = ngrams(word_tokenize(string),2)

for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Trigram

Trigram an n-gram consisting of three items of a sequence

- Three words that are used together to mean something specific

Method 1

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = nltk.trigrams(word_tokenize(string))

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

Method 2

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = ngrams(word_tokenize(string),3)

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

What is padding in NLP?

Padding is a technique used to ensure that all input sequences have the same length. This is necessary because neural networks require inputs that have the same shape and size. However, when we pre-process and use texts as inputs for our model, not all sentences have the same length. In other words, some sentences are longer or shorter than others. We need to have inputs with the same size, and this is where padding comes in.

Padding is a special form of masking where the masked steps are at the start or the end of a sequence. Padding comes from the need to encode sequence data into contiguous batches: in order to make all sequences in a batch fit a given standard length. The padding is added before splitting the sentence.

n-1 padding is added.

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize


string = "moses supposes his toeses are roses but moses supposes erroneously"

print('------------------- Padding in Bigrams -----------------------------')
```

```
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=2))
```

```
------------------- Padding in Bigrams ----------------------------
```

```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

```
print('------------------- Padding in Trigrams -----------------------------')
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=3))
```

```
------------------- Padding in Trigrams -----------------------------
```

```
[('<s>', '<s>', 'moses'),
 ('<s>', 'moses', 'supposes'),
 ('moses', 'supposes', 'his'),
 ('supposes', 'his', 'toeses'),
 ('his', 'toeses', 'are'),
 ('toeses', 'are', 'roses'),
 ('are', 'roses', 'but'),
 ('roses', 'but', 'moses'),
 ('but', 'moses', 'supposes'),
 ('moses', 'supposes', 'erroneously'),
 ('supposes', 'erroneously', '</s>'),
 ('erroneously', '</s>', '</s>')]
```

Alternatively, We can get rid of the parameters by using

> pad_both_ends method in NLTK

Note the n argument, that tells the function we need padding for bigrams.

```
from nltk.lm.preprocessing import pad_both_ends
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

list(ngrams(pad_both_ends(word_tokenize(string), n=2), n=2))
```

```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

Training the Maximum Likelihood Model

Maximum likelihood estimation estimates the model parameters such that the probability is maximized.

- Get the requisite n_grams frequency counts from a corpus.
- Normalize them to a 0-1 range.
- Build the vocabulary (of unique words): to create this vocabulary we need to pad our sentences (just like for counting ngrams) and then combine the sentences into one flat stream of words.

To do the above steps, we need to create an Ngram and add the padding then flatten the list. All these steps are requested for pre-processing to train the MLE model. However, we can use padded_everygram_pipeline() in NLTK to do all the steps in one call.

The padded_everygram_pipeline() will return:
1- **ngrams_gen:** Each sentence becomes a list of n-grams with padding.
2- **vocab_gen:** A flat stream of all tokens, with start and end symbols included

References:

https://www.nltk.org/api/nltk.lm.html

https://www.kaggle.com/code/alvations/n-gram-language-model-with-nltk

```
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from nltk.lm import smoothing

string = "moses supposes his toeses are roses but moses supposes erroneously"

#Create Bigram Language Model
train, vocab = padded_everygram_pipeline(2, [word_tokenize(string)])
```

```
from nltk.lm import MLE
lm = MLE(2)
lm.fit(train, vocab) #Train the Maximum Likelihood Model
```

Calculating Probabilities

Here's how you get the score for a word given some preceding context. For example we want to know what is the chance that "b" is preceded by "a".

```
print('P(supposes | moses) =', lm.score('supposes', ['moses']))
print('P(moses | supposes) =', lm.score('moses', ['supposes']))
```

```
P(supposes | moses) = 1.0
P(moses | supposes) = 0.0
```

Evaluating n_grams models using preplexity

Preplexity is a measurement of how well a probability model (like a language model) predicts a sample.

It's the probability of the test set normalized by the number of words.

A `lower` perplexity means:

- The model is less “perplexed” by the data.
- The model assigns higher probabilities to the actual words in the test set.
- Therefore, better performance.

A `higher perplexity` = more uncertainty = worse model.

```
test = [('moses', 'supposes'), ('toeses','are')]
print(lm.perplexity(test)) #Perplexity for bigram model
```

```
1.0
```

Task: Generating tweets using n_grams

In this Task, we will build a MLE model with n-gram to generate tweets.

The agenda is:

1. load the data: https://www.kaggle.com/datasets/adizafar/large-random-tweets-from-pakistan
2. pre-processing the data:

- Remove hashtags
- Remove RT
- Remove any website
- Remove any mentions
- Remove emojis

3. build MLE model with n-gram (Bi-gram)
4. evaluate the model
5. calculate the probability of the bigram: (pakistan is)
6. calculate the preplexity of the word: (pakistan)

before we start we need to install library to handle the emojis in tweets

> pip install emoji

```
import re, emoji, math
from sklearn.model_selection import train_test_split
from nltk.lm import MLE, Laplace
from nltk.lm.preprocessing import padded_everygram_pipeline, pad_both_ends
from nltk.util import ngrams

tweets = pd.read_csv(DATA / 'pakistan_tweets.csv', encoding='utf-8', encoding_errors='replace')
def clean_tweet(text):
    text = re.sub(r'https?://\S+|www\.\S+', ' ', str(text))
    text = re.sub(r'(?<!\w)[#@]\w+', ' ', text)
    text = re.sub(r'\bRT\b', ' ', text, flags=re.I)
    text = emoji.replace_emoji(text, replace=' ')
    return re.sub(r'\s+', ' ', text).strip().lower()

cleaned = tweets['full_text'].dropna().map(clean_tweet).drop_duplicates()
# English alphabet tokens define the vocabulary for this English bigram exercise.
sentences = [re.findall(r"[a-z]+(?:'[a-z]+)?", text) for text in cleaned]
sentences = [s for s in sentences if len(s) >= 2]
train_sents, test_sents = train_test_split(sentences, test_size=0.2, random_state=42)
tweet_lm = MLE(2)
train_ngrams, vocabulary = padded_everygram_pipeline(2, train_sents)
```

```
tweet_lm.fit(train_ngrams, vocabulary)
test_bigrams = [ng for sent in test_sents
                for ng in ngrams(pad_both_ends(sent,n=2),2)]
print('Raw tweets:',len(tweets), '| Usable unique tokenized tweets:',len(sentences))
print('Train/test sentences:',len(train_sents),len(test_sents))
print('Held-out bigram perplexity (MLE):',tweet_lm.perplexity(test_bigrams))
seen = [ng for ng in test_bigrams if tweet_lm.score(ng[-1],ng[:-1]) > 0]
print('Nonzero-probability bigrams:',len(seen),'of',len(test_bigrams))
print('Perplexity on seen bigrams only (diagnostic):',tweet_lm.perplexity(seen))
# Smoothing is a separately labeled comparison, not a replacement for required MLE.
smooth_lm = Laplace(2)
tr, voc = padded_everygram_pipeline(2,train_sents)
smooth_lm.fit(tr,voc)
print('Held-out bigram perplexity (Laplace):',smooth_lm.perplexity(test_bigrams))
print('P(is | pakistan):',tweet_lm.score('is',['pakistan']))
print('P(pakistan):',tweet_lm.score('pakistan'))
print('Single-token unigram perplexity:',tweet_lm.perplexity([('pakistan',)]))
print('Boundary-aware perplexity for the one-word sentence:',
      tweet_lm.perplexity(list(ngrams(pad_both_ends(['pakistan'],n=2),2))))
print('\nGenerated tweets:')
for seed in [7, 42, 123]:
    generated = tweet_lm.generate(25,text_seed=['<s>'],random_seed=seed)
    generated = generated[:generated.index('</s>')] if '</s>' in generated else generated
    print(' '.join(w for w in generated if w != '<s>'))
```

```
C:\Users\Azoz\AppData\Local\Temp\ipykernel_11332\836732453.py:7: DtypeWarning: Columns (5: reply_count) have
mixed types. Specify dtype option on import or set low_memory=False.
  tweets = pd.read_csv(DATA / 'pakistan_tweets.csv', encoding='utf-8', encoding_errors='replace')
```

```
Raw tweets: 202202 | Usable unique tokenized tweets: 96241
Train/test sentences: 76992 19249


Held-out bigram perplexity (MLE): inf


Nonzero-probability bigrams: 220604 of 329682


Perplexity on seen bigrams only (diagnostic): 88.20707361184465


Held-out bigram perplexity (Laplace): 10466.207205870267
P(is | pakistan): 0.04519774011299435
P(pakistan): 0.005128550597715187
Single-token unigram perplexity: 194.98686440677972
Boundary-aware perplexity for the one-word sentence: 30.178216193115716

Generated tweets:


how can put all pakistani gained in one
phander lake gilgit baltistan pakistan on train alone bewildered far as my all ends price million covid pandemic
the academic twitter open from childhood when


amp bars for autumn when am much energy star's most experienced haricane at
```

Interpretation

The required MLE has no smoothing, so a single unseen test bigram makes the complete test perplexity infinite. The seen-only diagnostic excludes hard cases and is not a valid replacement for the full test score. Laplace smoothing assigns nonzero probability to unseen transitions and is reported separately.

For the requested bigram `(pakistan is)`, the conditional probability is `P(is | pakistan)`. Perplexity belongs to a sequence of n-grams rather than a bare word: the single-token unigram result is `1/P(pakistan)`, while treating `pakistan` as a whole sentence also scores its start and end transitions. A bigram generator only remembers one previous token, so generated text may be locally plausible but incoherent. The corpus is multilingual; this English exercise explicitly discards non-Latin word tokens.